WLFI Unlock Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Kage

SBSecurity (Blckhv, Slavcheww)

May 6, 2026

Contents

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

World Liberty Financial is a protocol previously audited by Cyfrin featuring: * USD1, a US dollar stablecoin designed for stability, security, and transparency that users can borrow against collateral * WLFI Markets (powered by Dolomite), where users can supply assets to earn rewards or borrow using collateral * Governance via holders of the WLFI token, who can propose, review, and vote on the protocol's future

5 Audit Scope

The context for the audit was this governance proposal with one exception: "the election window of 10 days is being removed and will not be what's voted on".

The audit scope was limited to the unlock-related upgrade functionality introduced in PR8:

```
// two new functions wlfiBurnAllocation, wlfiSetCategory
contracts/wlfi/WorldLibertyFinancialRegistry.sol
contracts/wlfi/WorldLibertyFinancialVester.sol

// new file
contracts/wlfi/WorldLibertyFinancialV3.sol

// deployment script to be used for upgrade
script/wlfi/deploy-wlfi-v3-updates.ts

// also had some changes
script/wlfi/deploy-wlfi-v2_1-updates.ts
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the Unlock code provided by WLFI. The findings consist of 2 Medium & 8 Low severity issues with the remainder being informational and gas optimizations.

World Liberty Financial is a governance-driven DeFi protocol whose ecosystem includes the WLFI governance token, the USD1 stablecoin, and a borrow/lend market. This audit covered the V3 unlock upgrade, which implements the early-supporter, founder, team, and partner token unlock approved in the governance proposal. The upgrade introduces a one-shot "election" that lets legacy WLFI holders opt into a new vesting curve — retail tranches unlock without penalty, while team tranches incur a permanent 10% burn of the elector's allocation. Election can be triggered either by the user, via an off-chain authorised signature, or by the protocol owner on behalf of activated legacy accounts.

The most material findings centred on the election lifecycle and its admin-controlled boundaries. The whitelist agent could insert users directly into finalized categories (`45/47`), allowing the 10% team burn to be silently bypassed; the per-category `enabled` flag was not enforced in the claim path, so disabling a category had no effect on users already assigned to it; `wlfiBurnAllocation` could revert with a `Panic(0x01)` on high-claim users, DoS'ing bulk team elections; `ownerSetCategoryTemplate` did not validate that template percentages summed to 100%, allowing allocation to be permanently stranded by a single misconfigured setter; legacy-user re-inserts could reset `isActivated=false`, freezing the affected wallet via V2's transfer gate; and the team election path could discard a user's prior vesting templates and create a post-election claim blackout. **All major issues were fixed by World Liberty Financial and verified by Cyfrin as part of the mitigation review.**

A note on centralization risks of the contracts in scope - V3 unlock upgrade is highly admin-driven. The WLFI owner can force-elect any activated legacy account into the new categories (with the 10% burn applied to team tranches) without user consent, can reshape the unlock curves of categories that users have already elected into by editing templates at any time, and — together with the proxy admins — can swap the underlying implementations of WLFI, the Registry, and the Vester. A whitelist agent controls the legacy-user roster, an off-chain `authorizedSigner` key authorises user self-election, and a guardian role can pause the Vester and block all elections. The protocol is therefore fully upgradeable and fully pausable: holders are trusting the WLFI ownership and operations roles to act in good faith and to manage key custody securely throughout the lifetime of the unlock.

Summary

Project Name	Unlock
Repository	usd1-protocol
Commit	63caed8e682d...
Fix Commit	1430e2453497...
Audit Timeline	Apr 23rd - Apr 24th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	8
Informational	7
Gas Optimizations	1
Total Issues	18

Summary of Findings

[M-1] `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` accepts finalized categories (45, 47) allowing the whitelist agent to bypass the 10% V3 election burn	Resolved
[M-2] `ownerSetCategoryEnabled` disable is not enforced in the claim flow, allowing users in a disabled category to keep claiming	Resolved
[L-1] `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` can be griefed by dust transfers when WLFI is transferable	Acknowledged
[L-2] `WorldLibertyFinancialV3::_electVestingUpdate` does not verify `Vester` initialization, letting owner skip the 10% burn	Resolved
[L-3] `WorldLibertyFinancialVester::wlfiBurnAllocation` asserts `claimed <= allocation` post-burn, panicking on users who claimed more than 90% of their pre-election allocation	Acknowledged
[L-4] Linear vesting accrues from `startTimestamp` instead of `cliffTimestamp`, causing a phantom unlock at the cliff when `start < cliff`	Acknowledged
[L-5] Team election to category 47 discards the user's prior vesting templates and creates a post-election claim blackout	Resolved
[L-6] Legacy retail template is copied into category 45 with no validation of percentage or end timestamp	Resolved
[L-7] `WorldLibertyFinancialVester::ownerSetCategoryTemplate` does not enforce sum of `percentageOfAllocation` across templates, allowing misconfiguration to permanently strand user allocation	Resolved
[L-8] `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` overwrite resets `isActivated=false`, freezing the user's WLFI balance via V2's `_update` gate	Resolved
[I-1] Duplicated `WLFI` access control check can be extracted into a modifier	Resolved
[I-2] `WorldLibertyFinancialRegistry::wlfiBurnAllocation` leaves stale legacy-user storage and emits the wrong event when the full allocation is burned	Resolved
[I-3] `WorldLibertyFinancialV3::_electVestingUpdate` hard-codes "not category 1 $\rightarrow$ team" and hard-codes magic category numbers	Resolved

[I-4] Redundant `assert` in `WorldLibertyFinancialV3::_electVestingUpdate` after self-write	Resolved
[I-5] `assert` is used to enforce user-reachable invariants in `WorldLibertyFinancialVester`	Acknowledged
[I-6] WLFI pause does not block retail elections, only team elections and Vester-gated flows	Resolved
[I-7] `WorldLibertyFinancialV3::electVestingUpdate` signature has no nonce and no deadline, making replay protection purely state-based	Partially Resolved
[G-1] Public view functions that are never called internally can be marked `external`	Resolved

7 Findings

7.1 Medium Risk

7.1.1 `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` accepts finalized categories (45, 47) allowing the whitelist agent to bypass the 10% V3 election burn

Description: `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` does not validate that `_categories[i]` belongs to the set of legitimate "source" categories (1..~20). The whitelist agent (or owner) can insert a user directly with `_categories[i] == 45` or `_categories[i] == 47` — the **finalized** categories that are meant to be reachable only via `WorldLibertyFinancialV3::_electVestingUpdate`, which applies a 10% burn for team tranches.

```
// WorldLibertyFinancialRegistry.sol
function agentBulkInsertLegacyUsers(
    uint256 _expectedNonce,
    address[] calldata _users,
    uint256[] calldata _amounts,
    uint8[] calldata _categories
) external onlyWorldLibertyOwnerOrWhitelist(msg.sender) {
    ...
    for (uint256 i; i < _users.length; ++i) {
        if (_users[i] == address(0) || _amounts[i] == 0) {
            revert InvalidBulkInsertLegacyUserAtIndex(i);
        }
        if (WLFI.balanceOf(_users[i]) != _amounts[i]) {
            revert InvalidBulkInsertLegacyUserBalance(_users[i]);
        }

        $.legacyUserMap[_users[i]] = LegacyUser({
            amount: uint112(_amounts[i]),
            category: _categories[i], // @audit no validation - accepts 45 or 47
            isActivated: false
        });
        ...
    }
    ...
}
```

Because the V3 deploy script enables categories 45 and 47 (`ownerSetCategoryEnabled(45/47, true)`), `WorldLibertyFinancialVester::_activateVest` accepts them:

```
// File: contracts/wlfi/WorldLibertyFinancialVester.sol
function _activateVest(address _user, uint8 _category, uint112 _amount) internal {
    if (_user == address(0) || _category == 0 || _amount == 0) {
        revert InvalidParameters();
    }
    VesterStorage storage $ = _getStorage();
    if (!$.categoryInfo[_category].enabled) { // @audit does not reject 45/47
        revert CategoryNotEnabled(_category);
    }
    ...
    userInfo.category = _category;
    userInfo.allocation = _amount;
    userInfo.initialized = true;
    ...
}
```

Attack flow:

1. Agent calls `agentBulkInsertLegacyUsers(nonce, [victim], [A], [47])` — legitimate WLFI balance check passes.

2. `WorldLibertyFinancialV2::ownerActivateAccount(victim, false)` or `activateAccount(sig)` runs — Vester is initialized with `category = 47` and **full allocation** `A`.
3. V3's `_electVestingUpdate(victim)` can never be invoked because line 86 reverts with `Election-AlreadyPerformed` (category already 47). The 10% burn the protocol expected is **permanently skipped**.

Impact: The whitelist agent can bypass the economic mechanism of the V3 unlock described in the governance proposal. For a single team-tranche user with `allocation == 3_750_000_000e18` (the `teamSigner` referenced in `deploy-wlfi-v3-updates.ts`), this skips a burn of **375,000,000 WLFI**.

At scale across the team/founder/partner tranches, the unburned amount is a multi-billion WLFI delta — a direct violation of the governance-approved supply reduction.

Proof of Concept: Add the following test:

```
import { loadFixture } from '@nomicfoundation/hardhat-network-helpers';
import { expect } from 'chai';
import { parseEther } from 'ethers';
import { impersonateReq } from '../../script/execution-utils';
import { ONE_ETH_BI, ZERO_BI } from '../../src/no-dependencies-constants';
import { IWorldLibertyFinancialVester } from '../../src/types';
import { FinalizedVestingCategory, WLFI_START_TIMESTAMP } from '../../src/wlfi-constants';
import { deployWlfiV2Fixture } from '../fixtures';
import { expectEvent, expectThrowWithCustomError } from '../utils';
import { advanceTimeToAfterStartTimestamp } from '../wlfi/wlfi-utils';

type Ctx = Awaited<ReturnType<typeof deployWlfiV2Fixture>>;
let ctx: Ctx;

const DEFAULT_AMOUNT = parseEther('100');

const IMMEDIATE_TEMPLATE: IWorldLibertyFinancialVester.TemplateStruct = {
  percentageOfAllocation: parseEther('1'),
  startTimestamp: ZERO_BI,
  cliffTimestamp: ZERO_BI,
  endTimestamp: ZERO_BI,
};

describe('WorldLibertyFinancialRegistry - Edge Cases (audit)', () => {
  beforeEach(async () => {
    ctx = await loadFixture(deployWlfiV2Fixture);
  });

  describe('agentBulkInsertLegacyUsers  pre-plant finalized category (Finding [*Duplicated `WLFI` access
  ↪  control check can be extracted into a
  ↪  modifier*](#duplicated-wlfi-access-control-check-can-be-extracted-into-a-modifier))', () => {
    it('accepts _categories[i]=47 and allows user to activate into cat 47 at full allocation with ZERO
    ↪  burn', async function () {
      // enable cat 47 in Vester (as V3 deploy script does)
      const finalizedCat = Number(FinalizedVestingCategory.TEAM); // 47
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(finalizedCat, true);

      // Victim user has exact WLFI balance matching the allocation to be recorded.
      // hhUser2 starts with 0 WLFI (fixture funds hhUser1 only), so balance-match check aligns.
      const victim = ctx.core.hhUser2;
      await ctx.wlfi.connect(ctx.wlfiOwner).transfer(victim.address, DEFAULT_AMOUNT);

      // Agent (or compromised owner) inserts with the FINALIZED category
      const nonce = await ctx.registry.nonce();
      await ctx.registry.connect(ctx.wlfiOwner).agentBulkInsertLegacyUsers(
        nonce,
        [victim.address],
        [DEFAULT_AMOUNT],
```

```
        [finalizedCat],
      );

      // Snapshot WLFI totalSupply before activation
      const totalSupplyBefore = await ctx.wlfi.totalSupply();

      // Legitimate activation flow - owner or user-sig path
      await ctx.wlfi.connect(ctx.wlfiOwner).ownerActivateAccount(victim.address, false);

      // user activated into cat 47 with FULL allocation
      expect(await ctx.registry.getLegacyUserCategory(victim.address), 'Registry
      ↪  cat').to.eq(finalizedCat);
      expect(await ctx.registry.getLegacyUserAllocation(victim.address), 'Registry
      ↪  amount').to.eq(DEFAULT_AMOUNT);
      expect(await ctx.vester.allocation(victim.address), 'Vester allocation').to.eq(DEFAULT_AMOUNT);

      // @audit total WLFI supply unchanged - no 10% burn was ever applied
      expect(await ctx.wlfi.totalSupply(), 'totalSupply').to.eq(totalSupplyBefore);

      // Subsequent attempt to elect via V3 would revert ElectionAlreadyPerformed,
      // so the bypass is permanent - user got finalized terms without paying the burn.
    });
  });
});
```

Recommended Mitigation: Consider rejecting finalized categories at the Registry's bulk-insert boundary. Alternatively, whitelist only the known base categories (e.g., 1..20) explicitly, which also rejects typos and stray values such as category `0`.

```
function agentBulkInsertLegacyUsers(
        uint256 _expectedNonce,
        address[] calldata _users,
        uint256[] calldata _amounts,
        uint8[] calldata _categories) external onlyWorldLibertyOwnerOrWhitelist(msg.sender) {
{
  ...
  for (uint256 i; i < _users.length; ++i) {
++    if (_categories[i] == 45 || _categories[i] == 47) {
++        revert InvalidBulkInsertLegacyUserAtIndex(i);
++   }
        ...
  }
}
```

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.1.2 `ownerSetCategoryEnabled` disable is not enforced in the claim flow, allowing users in a disabled category to keep claiming

Description: `ownerSetCategoryEnabled` toggles the per-category `enabled` flag, but the flag is only checked on entry-point functions that *assign* a user to a category — `_activateVest` and `wlfiSetCategory`:

```
//WorldLibertyFinancialVester.sol#L44-L47
function ownerSetCategoryEnabled(uint8 _category, bool _enabled) external
↪  onlyWorldLibertyOwner(msg.sender) {
    _getStorage().categoryInfo[_category].enabled = _enabled;
    emit SetCategoryEnabled(_category, _enabled);
}
```

```
//WorldLibertyFinancialVester.sol#L243-L245
function _activateVest(address _user, uint8 _category, uint112 _amount) internal {
    ...
    if (!$.categoryInfo[_category].enabled) {
        revert CategoryNotEnabled(_category);
    }
    ...
}
```

```
//WorldLibertyFinancialVester.sol#L164-L166
function wlfiSetCategory(address _user, uint8 _category) external whenNotPaused {
    ...
    if (!$.categoryInfo[_category].enabled) {
        revert CategoryNotEnabled(_category);
    }
    ...
}
```

The claim path — `_claim` → `_claimable` → `_unlockedTotal` — never reads `$.categoryInfo[_category].enabled`. It only consumes `templateCount` and `categoryTemplates[_category][i]`. Once a user's `UserInfo.category` has been set at activation (when `enabled` was true), the `enabled` flag becomes meaningless for their ongoing claims.

Impact: The `enabled` flag does not do what its name and natspec imply:

- Disabling a category via `ownerSetCategoryEnabled(_category, false)` has no effect on any user already assigned to that category; they continue to accrue unlocked amounts according to the templates and can freely call `wlfiClaimFor`.

Recommended Mitigation: Enforce the `enabled` flag in the claim flow. The cleanest place is `WorldLibertyFinancialVester::_claimable`, so both external `claimable` views and `_claim` honor it automatically:

```
    function _claimable(
        VesterStorage storage $,
        UserInfo memory _userInfo
    ) internal view returns (uint256) {
        if (!_userInfo.initialized) {
            return 0;
        }
+       if (!$.categoryInfo[_userInfo.category].enabled) {
+           return 0;
+       }

        uint256 unlocked = _unlockedTotal($, _userInfo.category, _userInfo.allocation);
        return unlocked > _userInfo.claimed ? unlocked - _userInfo.claimed : 0;
    }
```

If a hard revert is preferred over a silent zero, add the same check in `_claim` and revert with `CategoryNotEnabled(_userInfo.category)` so users get a clear error. Either way, the flag should be enforced consistently so admins have a per-category circuit breaker independent of the global pause.

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.2 Low Risk

7.2.1 `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` can be griefed by dust transfers when WLFI is transferable

Description: In `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers`, the operator supplies a list of `_users` and `_amounts`, and each row is checked with strict equality against the user's current WLFI balance.

```
   function agentBulkInsertLegacyUsers(
        uint256 _expectedNonce,
        address[] calldata _users,
        uint256[] calldata _amounts,
        uint8[] calldata _categories
    ) external onlyWorldLibertyOwnerOrWhitelist(msg.sender) {
  ...
  for (uint256 i; i < _users.length; ++i) {
    ...
    if (WLFI.balanceOf(_users[i]) != _amounts[i]) {
        revert InvalidBulkInsertLegacyUserBalance(_users[i]);
    }
    ...
 }
}
```

The whole batch reverts if any single row fails this check. If WLFI is transferable at the time the operator runs this function, any external address can send a small amount of WLFI to any `_users[i]` to change its balance and force the batch to revert. The target user can also self grief by transferring their own balance out.

Impact: No funds are at risk and the nonce does not advance on revert, so the operator can retry. The result is wasted gas and a broken batching workflow: the operator must rebuild the snapshot, rebuild the arrays, and resubmit. A motivated griefer that watches the mempool can repeat this each time the operator tries to insert legacy users.

Proof of Concept: Assume WLFI is transferable and the operator has prepared a batch that includes `alice` with `_amounts[i] = 1000e18` matching `alice`'s current balance.

1. Operator submits the tx with a batch containing `alice`.
2. An attacker sees the pending tx in the mempool and frontruns it by sending `1 wei` of WLFI to `alice`.
3. The operator's tx executes. When the loop reaches `alice`, `WLFI.balanceOf(alice)` returns `1000e18 + 1` which is not equal to `_amounts[i]`.
4. The tx reverts with `InvalidBulkInsertLegacyUserBalance(alice)` and the full batch is lost.

The same outcome occurs if `alice` herself transfers any amount out before the tx mines.

Recommended Mitigation: Consider skipping the affected row instead of reverting the full batch, and emit an event so the operator can reconcile it later.

WLFI: Acknowledged. We don't plan on using this function anymore and to the extent we even need to, we'd rather it not fail silently for some users.

7.2.2 `WorldLibertyFinancialV3::_electVestingUpdate` does not verify `Vester` initialization, letting owner skip the 10% burn

Description: `WorldLibertyFinancialV3::_electVestingUpdate` determines the 10% burn amount by reading `VESTER.allocation(_account)`. If the Vester has no record for the user, that view returns `0`, so the computed burn amount is `0 / 10 = 0`.

Both `VESTER.wlfiBurnAllocation` and `REGISTRY.wlfiBurnAllocation` then no-op silently (no `initialized` check on Vester's side), and `wlfiSetCategory` flips the category to 47 on both contracts without any token movement.

This "no Vester record despite Registry activation" state is reachable through the `_bypassVester=true` branch in V2's `_activateAccount`:

```
// WorldLibertyFinancialV2.sol
function _activateAccount(address _account, bool _bypassVester) internal {
    REGISTRY.wlfiActivateAccount(_account);
    uint8 category = REGISTRY.getLegacyUserCategory(_account);
    uint112 allocation = REGISTRY.getLegacyUserAllocation(_account);

    if (!_bypassVester) { // @audit when true, skips Vester.wlfiActivateVest
        _approve(_account, address(VESTER), 0);
        _approve(_account, address(VESTER), allocation);
        VESTER.wlfiActivateVest(_account, category, allocation);
        assert(allowance(_account, address(VESTER)) == 0);
    }
}
```

`WorldLibertyFinancialV3::_electVestingUpdate` then trusts Registry's `isActivated = true` and proceeds even though Vester state is empty:

```
// WorldLibertyFinancialV3.sol
function _electVestingUpdate(address _account) internal {
    if (!REGISTRY.isLegacyUser(_account)) {
        revert InvalidAccount();
    }
    if (REGISTRY.isLegacyUserAndIsNotActivated(_account)) { // @audit only checks Registry, not Vester
        revert AccountNotActivated(_account);
    }
    ...
    if (newCategory == 47) {
        uint256 allocation = VESTER.allocation(_account); // @audit returns 0 for non-initialized users
        uint256 amountToBurn = allocation / 10;            // @audit 0 / 10 = 0
        VESTER.wlfiBurnAllocation(_account, amountToBurn); // @audit no-op with _amount = 0
        REGISTRY.wlfiBurnAllocation(_account, amountToBurn); // @audit no-op with _amount = 0
    }
    VESTER.wlfiSetCategory(_account, newCategory); // @audit writes cat=47 on uninitialized Vester
    ↪  record
    REGISTRY.wlfiSetCategory(_account, newCategory);
    ...
}
```

`WorldLibertyFinancialVester::wlfiBurnAllocation` and `wlfiSetCategory` each omit an `initialized` check, completing the silent-no-op path:

```
// WorldLibertyFinancialVester.sol
function wlfiBurnAllocation(address _user, uint256 _amount) external whenNotPaused {
    if (msg.sender != address(WLFI)) { revert Unauthorized(); }
    VesterStorage storage $ = _getStorage();
    UserInfo storage userInfo = $.users[_user];
    userInfo.allocation -= uint112(_amount); // @audit 0 - 0 = 0 no-op
    assert(userInfo.claimed <= userInfo.allocation);
    ...
}

function wlfiSetCategory(address _user, uint8 _category) external whenNotPaused {
    if (msg.sender != address(WLFI)) { revert Unauthorized(); }
    VesterStorage storage $ = _getStorage();
    if (!$.categoryInfo[_category].enabled) { revert CategoryNotEnabled(_category); }
    UserInfo storage userInfo = $.users[_user];
    // @audit no `initialized` check - writes category on empty record
    uint8 oldCategory = userInfo.category;
    userInfo.category = _category;
    ...
```

```
}
```

Attack flow:

1. Owner calls `V2.ownerActivateAccount(teamUser, true)` — Registry flips `isActivated=true`; Vester remains untouched (no record, no token pull).
2. Owner calls `V3.ownerElectVestingUpdatesFor([teamUser])`. V3 reads `VESTER.allocation(teamUser) = 0`, computes `amountToBurn = 0`, "burns" zero, and flips category to 47 on both contracts.
3. `teamUser` now holds 100% of their WLFI as a fully-liquid balance (Vester has no custody), Registry marks them as cat-47 "elected". **Zero WLFI was burned.**
4. Any attempt to subsequently call `V2.ownerActivateAccount(teamUser, false)` reverts with `AlreadyInitialized` (Registry's `isActivated` already flipped at step 1), so the user CANNOT be pulled into the Vester afterwards. The state is "one-way": team user retains 100% liquid.

Impact: Strictly worse than a normal team election. A normal team user pays 10% burn and receives 90% linearly vested over 3 years (Vester custody). A "bypass" team user pays 0% burn and retains 100% fully liquid immediately. For the `teamSigner` reference in the V3 deploy script (`3_750_000_000e18` allocation), this turns a `0.375e27` burn into `0` and lets the user transfer `3.75e27` WLFI immediately instead of waiting 3 years.

Proof of Concept: Run the following test:

```
import { loadFixture } from '@nomicfoundation/hardhat-network-helpers';
import { expect } from 'chai';
import { parseEther, TypedDataDomain } from 'ethers';
import hardhat from 'hardhat';
import { deployWlfiV3Fixture } from '../fixtures';
import { expectThrowWithCustomError } from '../utils';
import { advanceTimeToAfterStartTimestamp, signWlfiActivationMessage } from '../wlfi/wlfi-utils';

type Ctx = Awaited<ReturnType<typeof deployWlfiV3Fixture>>;
let ctx: Ctx;


async function insertAndFund(user: { address: string }, amount: bigint, category: number) {
  await ctx.wlfi.connect(ctx.wlfiOwner).transfer(user.address, amount);
  const nonce = await ctx.registry.nonce();
  await ctx.registry.connect(ctx.wlfiOwner).agentBulkInsertLegacyUsers(
    nonce,
    [user.address],
    [amount],
    [category],
  );
}

describe('WorldLibertyFinancialV3 - Edge Cases (audit)', () => {
  beforeEach(async () => {
    ctx = await loadFixture(deployWlfiV3Fixture);

    // Enable categories used across tests (1, 2 for pre-election; 45, 47 for post-election)
    await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(1, true);
    await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(2, true);
    await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(45, true);
    await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(47, true);

    await advanceTimeToAfterStartTimestamp(ctx);
  });


  describe('bypass-vester', () => {
    it('elects a team user into cat 47 with zero burn when owner uses ownerActivateAccount(user,
    ↪  bypassVester=true)', async function () {
```

```
      // team user (cat 2) registered with 100 WLFI; no Vester activation
      const user = ctx.core.hhUser2;
      const amount = parseEther('100');
      await insertAndFund(user, amount, 2);

      const userBalanceBefore = await ctx.wlfi.balanceOf(user.address);
      const vesterBalanceBefore = await ctx.wlfi.balanceOf(ctx.vester);
      const totalSupplyBefore = await ctx.wlfi.totalSupply();

      // Owner calls the bypass-vester activation path: Registry.isActivated flips to true,
      // Vester is NEVER initialized, Vester.allocation(user) remains 0.
      await ctx.wlfi.connect(ctx.wlfiOwner).ownerActivateAccount(user.address, true);

      // owner elects this bypass-activated user
      await ctx.wlfi.connect(ctx.wlfiOwner).ownerElectVestingUpdatesFor([user.address]);

      //no tokens moved, no burn occurred
      expect(await ctx.wlfi.balanceOf(user.address), 'user WLFI balance').to.eq(userBalanceBefore);
      expect(await ctx.wlfi.balanceOf(ctx.vester), 'Vester WLFI balance').to.eq(vesterBalanceBefore);
      expect(await ctx.wlfi.totalSupply(), 'WLFI total supply').to.eq(totalSupplyBefore);

      // Vester has no record (allocation stayed 0), but category was set to 47
      expect(await ctx.vester.allocation(user.address), 'Vester.allocation').to.eq(0n);
      expect(await ctx.vester.unclaimed(user.address), 'Vester.unclaimed').to.eq(0n);

      // Registry reports cat 47 and amount UNCHANGED (since burn amount was 0)
      expect(await ctx.registry.getLegacyUserCategory(user.address), 'Registry cat').to.eq(47);
      expect(await ctx.registry.getLegacyUserAllocation(user.address), 'Registry amount').to.eq(amount);

      // user can still freely transfer WLFI (not a legacy-not-activated user)
      const sink = ctx.core.hhUser3;
      const transferTx = ctx.wlfi.connect(user).transfer(sink.address, amount);
      await expect(transferTx).to.not.be.reverted;
      expect(await ctx.wlfi.balanceOf(sink.address)).to.eq(amount);
    });
  });


});
```

Recommended Mitigation: Consider adding a Vester-state check at the top of `_electVestingUpdate`:

```
function _electVestingUpdate(address _account) internal {
    if (!REGISTRY.isLegacyUser(_account)) { revert InvalidAccount(); }
    if (REGISTRY.isLegacyUserAndIsNotActivated(_account)) { revert AccountNotActivated(_account); }

++    if (VESTER.allocation(_account) == 0) { revert VesterNotInitialized(_account); }

    ...
}
```

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.2.3 `WorldLibertyFinancialVester::wlfiBurnAllocation` asserts `claimed <= allocation` post-burn, panicking on users who claimed more than 90% of their pre-election allocation

Description: `WorldLibertyFinancialVester::wlfiBurnAllocation` decrements `userInfo.allocation` by the requested amount, then asserts `userInfo.claimed <= userInfo.allocation`. When called during V3's team election, the burn amount is `allocation / 10`, so the post-burn allocation is `0.9 * old_allocation`. If the user

has claimed more than that post-burn threshold (i.e., `claimed > 0.9 * old_allocation`), system panics.

```
// WorldLibertyFinancialVester.sol
function wlfiBurnAllocation(address _user, uint256 _amount) external whenNotPaused {
    if (msg.sender != address(WLFI)) { revert Unauthorized(); }
    VesterStorage storage $ = _getStorage();
    UserInfo storage userInfo = $.users[_user];
    userInfo.allocation -= uint112(_amount); // post-burn: 0.9 * old_allocation
    assert(userInfo.claimed <= userInfo.allocation); // @audit panics when claimed > 0.9*old_allocation
    $.totalAllocated -= uint112(_amount);
    assert($.totalClaimed <= $.totalAllocated);
    WLFI.burn(_amount);
    ...
}
```

A single such user in a batched array aborts the entire transaction — rolling back the successful elections of every other user in the array.

```
// WorldLibertyFinancialV3.sol
function ownerElectVestingUpdatesFor(address[] calldata _accounts) external onlyOwner {
    for (uint256 i; i < _accounts.length; ++i) {
        _electVestingUpdate(_accounts[i]); // @audit any revert here aborts the entire loop
    }
}
```

For the user in isolation (user-path `electVestingUpdate(sig)` or owner-path on a single address), the same panic blocks the election entirely — the user cannot transition to cat 47 via any path.

As per scope, the governance proposal's 10-day election cap was removed — elections are open indefinitely with only state-based replay protection (`ElectionAlreadyPerformed`). Without a deadline, any user who eventually claims >90% of their pre-election allocation becomes permanently un-electable and permanently poisons any batch that includes them. The bug's population grows monotonically as long as old-category templates continue vesting and elections remain pending.

Impact: A high-claimed team user cannot be elected into cat 47 at all — not by the owner, not by their own signed message. Also, a single "poisoned" user in a batch of 500 aborts the entire batch. The owner must identify the offender off-chain (from the bare `Panic(0x01)` — no custom error payload identifies the user), remove them, and retry.

Proof of Concept: Run the following test:

```
import { loadFixture, time } from '@nomicfoundation/hardhat-network-helpers';
import { expect } from 'chai';
import { parseEther } from 'ethers';
import { impersonateReq } from '../../script/execution-utils';
import { ONE_ETH_BI, ZERO_BI } from '../../src/no-dependencies-constants';
import { IWorldLibertyFinancialVester } from '../../src/types';
import { FinalizedVestingCategory, WLFI_START_TIMESTAMP } from '../../src/wlfi-constants';
import { deployWlfiV2Fixture } from '../fixtures';
import { expectEvent, expectThrowWithCustomError } from '../utils';
import { advanceTimeToAfterStartTimestamp } from '../wlfi/wlfi-utils';

type Ctx = Awaited<ReturnType<typeof deployWlfiV2Fixture>>;
let ctx: Ctx;

const DEFAULT_AMOUNT = parseEther('100');
const DEFAULT_CATEGORY = 2n;

// An immediate-unlock template: endTimestamp = 0 means "fully unlocked regardless of time"
const IMMEDIATE_TEMPLATE: IWorldLibertyFinancialVester.TemplateStruct = {
  percentageOfAllocation: parseEther('1'),
  startTimestamp: ZERO_BI,
  cliffTimestamp: ZERO_BI,
```

```
  endTimestamp: ZERO_BI,
};

describe('WorldLibertyFinancialVester - Edge Cases (audit)', () => {
  beforeEach(async () => {
    ctx = await loadFixture(deployWlfiV2Fixture);
  });


  describe('wlfiBurnAllocation assert panic on over-claimed user (Vester Finding [*Duplicated `WLFI`
  ↪  access control check can be extracted into a
  ↪  modifier*](#duplicated-wlfi-access-control-check-can-be-extracted-into-a-modifier))', () => {
    it('panics when userInfo.claimed > userInfo.allocation - _amount', async function () {
      const wlfi = await impersonateReq(ctx.wlfi, true);
      const user = ctx.core.hhUser1;

      // Set up cat 2 with immediate 100% unlock
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(Number(DEFAULT_CATEGORY), true);
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryTemplate(
        Number(DEFAULT_CATEGORY), 0, IMMEDIATE_TEMPLATE,
      );

      // Activate user with 100 WLFI
      await ctx.wlfi.connect(ctx.wlfiOwner).transfer(user.address, DEFAULT_AMOUNT);
      await ctx.wlfi.connect(user).approve(ctx.vester, DEFAULT_AMOUNT);
      await ctx.vester.connect(wlfi).wlfiActivateVest(user.address, Number(DEFAULT_CATEGORY),
      ↪  DEFAULT_AMOUNT);

      await advanceTimeToAfterStartTimestamp(ctx);

      // User claims all 100 (immediate unlock template)
      await ctx.vester.connect(wlfi).wlfiClaimFor(user.address);
      expect(await ctx.vester.claimed(user.address)).to.eq(DEFAULT_AMOUNT);
      expect(await ctx.vester.allocation(user.address)).to.eq(DEFAULT_AMOUNT);

      // Snapshot state that should be unwound by the panic
      const allocBefore = await ctx.vester.allocation(user.address);
      const totalAllocBefore = await ctx.vester.totalAllocated();
      const totalSupplyBefore = await ctx.wlfi.totalSupply();

      // try to burn 10% of allocation.
      //   userInfo.allocation = 100; claimed = 100
      //   post-sub: allocation = 90; assert(claimed=100 <= allocation=90) -> FALSE -> panic
      const burnAmount = DEFAULT_AMOUNT / 10n;
      await expect(
        ctx.vester.connect(wlfi).wlfiBurnAllocation(user.address, burnAmount),
      ).to.be.reverted;
    });

  });

});
```

Recommended Mitigation: Consider a two-layer fix — safe-burn math in the Vester, per-user failure isolation in V3 via a try-catch in a self-call mode:

```
// File: contracts/wlfi/WorldLibertyFinancialVester.sol
function wlfiBurnAllocation(address _user, uint256 _amount) external whenNotPaused {
    if (msg.sender != address(WLFI)) { revert Unauthorized(); }
    VesterStorage storage $ = _getStorage();
    UserInfo storage userInfo = $.users[_user];
```

```
++    uint112 amount112 = uint112(_amount);
++    if (uint256(userInfo.claimed) + uint256(amount112) > uint256(userInfo.allocation)) {
++       revert ClaimedExceedsPostBurnAllocation(_user);
++    }

    userInfo.allocation -= amount112;
    // (assert can remain as belt-and-suspenders, but is now unreachable)
    assert(userInfo.claimed <= userInfo.allocation);
    ...
}
```

```
// WorldLibertyFinancialV3.sol
++ event ElectVestingUpdateFailed(address indexed account, bytes errorData);

++ function _tryElectVestingUpdate(address _account) external {
++    require(msg.sender == address(this), "self-only");
++   _electVestingUpdate(_account);
++ }

++ function ownerElectVestingUpdatesFor(address[] calldata _accounts) external onlyOwner {
++   for (uint256 i; i < _accounts.length; ++i) {
++        try this._tryElectVestingUpdate(_accounts[i]) {
++           // success
++        } catch (bytes memory errorData) {
++            emit ElectVestingUpdateFailed(_accounts[i], errorData);
++       }
++    }
++}
```

WLFI: Acknowledged. There are no users who have claimed non-zero tokens.

7.2.4 Linear vesting accrues from `startTimestamp` instead of `cliffTimestamp`, causing a phantom unlock at the cliff when `start < cliff`

Description: `_segmentUnlocked` computes the linear unlock as a fraction of `(currentTimestamp - startTimestamp) / (endTimestamp - startTimestamp)`. The pre-cliff branch returns `0` while `currentTimestamp < cliffTimestamp`, but the linear schedule itself is anchored at `startTimestamp`, not `cliffTimestamp`.

```
//WorldLibertyFinancialVester.sol#L339-L365
function _segmentUnlocked(Template memory _template, uint256 _segmentCap) internal view returns
↪   (uint256) {
    assert(_segmentCap != 0);

    uint32 currentTimestamp = uint32(block.timestamp);

    if (_template.endTimestamp == 0) {
        return _segmentCap;
    }
    if (currentTimestamp < _template.cliffTimestamp) {
        return 0;
    }
    if (currentTimestamp >= _template.endTimestamp) {
        return _segmentCap;
    }

    uint256 elapsed = uint256(currentTimestamp) - uint256(_template.startTimestamp);
    uint256 span = uint256(_template.endTimestamp) - uint256(_template.startTimestamp);

    assert(span != 0);

    return (_segmentCap * elapsed) / span;
```

```
}
```

The template setter at `ownerSetCategoryTemplate` only enforces the ordering `startTimestamp <= cliffTimestamp <= endTimestamp`, so any template with `start < cliff` is accepted, and the interface explicitly documents the schedule as `linear from (startTimestamp -> endTimestamp); cliffTimestamp must be greater than or equal to startTimestamp`:

```
//WorldLibertyFinancialVester.sol#L49-L81
function ownerSetCategoryTemplate(
    uint8 _category,
    uint8 _index,
    Template calldata _template
) external onlyWorldLibertyOwner(msg.sender) {
    if (_index >= MAX_TEMPLATE_COUNT) {
        revert InvalidTemplateCount();
    }
    if (_template.endTimestamp != 0) {
        if (
            _template.startTimestamp > _template.cliffTimestamp
            || _template.cliffTimestamp > _template.endTimestamp
        ) {
            revert InvalidTemplateTimestamp();
        }
    }
    ...
}
```

When `start < cliff`, accrual silently builds between `start` and `cliff` while the pre-cliff guard hides it. The instant `block.timestamp` reaches `cliff`, `_claimable` jumps from `0` to `segmentCap * (cliff - start) / (end - start)` in a single block.

Impact: The proposal language for early-supporter style allocations reads as `2-year cliff, then 2-year linear vest, tokens beginning to unlock at year 2 and fully distributed by year 4`. The intuitive template configuration for that schedule is `start = electionTime`, `cliff = start + 2y`, `end = start + 4y`. With the current math the user receives `50%` of their allocation in the same block the cliff is crossed, instead of unlocking smoothly from `0%` at year 2 to `100%` at year 4. After 1 year past the cliff the user holds `75%` instead of the spec's `50%`.

The bug is dormant only as long as every template is configured with `start == cliff`. Nothing in the contract enforces that invariant and the interface docs explicitly allow `start < cliff`. Any team category created with the proposal's natural reading distributes more tokens earlier than the proposal's vesting curve and increases circulating supply ahead of schedule.

Proof of Concept: The following test was added to `test/wlfi/WorldLibertyFinancialVester.test.ts` as part of this audit and passes against the current code on a fork at block `23_200_000`.

Result on the current code:

```
WorldLibertyFinancialVester
  Edge Cases and Branch Coverage
    _segmentUnlocked
      Resetting hardhat fork for network ethereum...
      PHANTOM UNLOCK: jumps to 50% of allocation at cliff time when start < cliff

1 passing
```

The body of the PoC pins down the buggy values directly. Schedule under test: `start = electionTime`, `cliff = start + 2y`, `end = start + 4y`, allocation `100 WLFI`, single template at `100%`:

```
const start = WLFI_START_TIMESTAMP;
const cliff = WLFI_START_TIMESTAMP + TWO_YEARS_S;   // start + 2y
const end   = WLFI_START_TIMESTAMP + FOUR_YEARS_S;  // start + 4y
...
// Just before cliff: pre-cliff guard returns 0, as expected.
```

```
await time.increaseTo(cliff - 1n);
expect(await ctx.vester.claimable(ctx.core.hhUser1)).to.eq(0);

// AT cliff: spec implies 0% (start of linear). Bug returns 50%.
await time.increaseTo(cliff);
expect(await ctx.vester.claimable(ctx.core.hhUser1)).to.eq(amount / 2n);

// 1 year past cliff: spec says 50% (halfway through 2y linear). Bug returns 75%.
await time.increaseTo(cliff + (TWO_YEARS_S / 2n));
expect(await ctx.vester.claimable(ctx.core.hhUser1)).to.eq((amount * 3n) / 4n);
```

Recommended Mitigation: Anchor the linear schedule at `cliffTimestamp` so the cliff actually marks the start of vesting, not just a claim gate over a schedule that has been silently accruing since `startTimestamp`:

```
-         uint256 elapsed = uint256(currentTimestamp) - uint256(_template.startTimestamp);
-         uint256 span = uint256(_template.endTimestamp) - uint256(_template.startTimestamp);
+         uint256 elapsed = uint256(currentTimestamp) - uint256(_template.cliffTimestamp);
+         uint256 span = uint256(_template.endTimestamp) - uint256(_template.cliffTimestamp);
```

The `assert(span != 0)` invariant still holds: the pre-cliff guard plus the `currentTimestamp >= endTimestamp` early return together imply `cliff <= currentTimestamp < endTimestamp` at the point the assertion runs, hence `endTimestamp > cliffTimestamp` and `span > 0`. Reflipping the test assertions noted in the PoC test to `0` at cliff and `amount / 2n` at cliff + 1y after the fix is sufficient to lock the new behavior in.

If the team wants to preserve the option of templates whose linear schedule predates the cliff (so that the cliff intentionally releases a precomputed chunk), enforce `startTimestamp == cliffTimestamp` in `ownerSetCategoryTemplate` and update the interface docs accordingly, so the API can no longer be configured into the buggy regime.

WLFI: Acknowledged. This is expected behavior. To mitigate this, we purposefully set `startTimestamp == cliffTimestamp` when we want to avoid this behavior.

7.2.5 Team election to category 47 discards the user's prior vesting templates and creates a post-election claim blackout

Description: The retail path of `_electVestingUpdate` preserves the user's previous vesting by copying the old category-1 template into category 45 at index 0 before the new 2-year template is appended at index 1. The team path does not. After election, a category-47 user has a single template covering 100% of their (post-burn) allocation linearly from `VESTING_START_TIMESTAMP` to `VESTING_START_TIMESTAMP + THREE_YEARS_IN_SECONDS`, with no carryover of the prior schedule.

`UserInfo.claimed` is preserved across the category switch. Claimable amount in the vester is `unlocked - claimed`:

```
//WorldLibertyFinancialVester.sol#L285-L295
function _claimable(
    VesterStorage storage $,
    UserInfo memory _userInfo
) internal view returns (uint256) {
    if (!_userInfo.initialized) {
        return 0;
    }

    uint256 unlocked = _unlockedTotal($, _userInfo.category, _userInfo.allocation);
    return unlocked > _userInfo.claimed ? unlocked - _userInfo.claimed : 0;
}
```

The deploy script wires up the asymmetry: category 45 keeps the old template plus the new one, while category 47 only sets one template.

```
//deploy-wlfi-v3-updates.ts#L153-L196
// Bring over the old template
```

```
vesterProxy.ownerSetCategoryTemplate,
[
  FinalizedVestingCategory.RETAIL, 0, {
  percentageOfAllocation: oldTemplates[0].percentageOfAllocation,
  startTimestamp: oldTemplates[0].startTimestamp,
  cliffTimestamp: oldTemplates[0].cliffTimestamp,
  endTimestamp: oldTemplates[0].endTimestamp,
},
],
// Put in 2 + 2
vesterProxy.ownerSetCategoryTemplate,
[
  FinalizedVestingCategory.RETAIL, 1, {
  percentageOfAllocation: parseEther(`${0.8}`),
  startTimestamp: VESTING_START_TIMESTAMP,
  cliffTimestamp: VESTING_START_TIMESTAMP,
  endTimestamp: VESTING_START_TIMESTAMP + TWO_YEARS_IN_SECONDS,
},
],
// Put in 2 + 3, all allocation
vesterProxy.ownerSetCategoryTemplate,
[
  FinalizedVestingCategory.TEAM, 0, {
  percentageOfAllocation: parseEther(`${1}`),
  startTimestamp: VESTING_START_TIMESTAMP,
  cliffTimestamp: VESTING_START_TIMESTAMP,
  endTimestamp: VESTING_START_TIMESTAMP + THREE_YEARS_IN_SECONDS,
},
],
```

For any team-cohort user who was previously activated and has `claimed > 0` under their old category template, election produces:

- `allocation_new = 0.9 * allocation_old`
- `claimed` unchanged
- `unlocked_new(t) = 0.9 * allocation_old * (t - VESTING_START) / 3y`, clamped to `[0, 0.9 * allocation_old]`
- `claimable_new = max(0, unlocked_new(t) - claimed)`

`claimable_new` stays at 0 until `0.9 * allocation_old * (t - VESTING_START) / 3y > claimed`, i.e. for roughly `3y * claimed / (0.9 * allocation_old)` after the cliff. A team user who had already claimed 10% of their pre-burn allocation faces a ~4-month blackout past `VESTING_START` before any new claim succeeds.

Impact: The proposal advertises a 2-year cliff followed by a 3-year linear vest for the team cohort. For team users who claimed any tokens under their previous category, the actual post-election schedule is later than that: claims do not resume at the cliff, they resume only after the new linear curve has caught up to the user's already-claimed amount. This is not disclosed in the proposal text, is not exercised by the script's `invariants` block (which uses a team user with `claimed = 0`), and is asymmetric with the retail design that explicitly carries the old template forward to avoid this exact effect.

The asymmetry also means the proposal's "10% burn + 3-year linear" description is slightly misleading: a team user with prior claims effectively receives less than 90% of their tokens over the 3-year window because the early portion is consumed by their existing `claimed`. Allocation accounting is correct in aggregate, but the user-visible claim curve is not the one the proposal describes.

Proof of Concept: Steps with concrete numbers:

```
1. Pre-election (in category 18, e.g.):
   - allocation = 1000 WLFI
   - claimed    = 100 WLFI (user previously claimed 10% under old schedule)
2. Owner calls ownerElectVestingUpdatesFor([user]) at any time before VESTING_START.
```

```
   - Burn 10%: allocation -> 900 WLFI (assert claimed <= allocation passes: 100 <= 900)
   - Category set to 47 in both vester and registry.
3. At VESTING_START (cliff): unlocked = 0 (start == cliff, elapsed = 0).
   claimable = max(0, 0 - 100) = 0.
4. At VESTING_START + 4 months: unlocked = 900 * (4/36) = 100 WLFI.
   claimable = max(0, 100 - 100) = 0. Still cannot claim.
5. At VESTING_START + 4 months + 1 day: unlocked just exceeds 100, claimable becomes
   non-zero. The user only now starts seeing claims, despite the cliff having passed.
```

The retail flow does not exhibit this because template 0 of category 45 is the old template, so the previously-vested chunk that produced `claimed` is still represented in `unlocked` after election.

Recommended Mitigation: Either preserve the prior team template the same way retail does, or refuse to elect when the new schedule would produce `unlocked < claimed` at `VESTING_START`.

Preserving the prior template requires the script to read the user's previous category templates per legacy team category and copy them into category 47 alongside the new template. Because team users span multiple legacy categories (2 through 20), this is not a single template copy and may require category 47 to be split or for templates to be set per-user, which is incompatible with the current shared-category model.

A simpler fix is to enforce the precondition in `_electVestingUpdate`:

```
if (newCategory == 47) {
    uint256 allocation = VESTER.allocation(_account);
    uint256 amountToBurn = allocation / 10;
    uint256 newAllocation = allocation - amountToBurn;
    uint256 alreadyClaimed = VESTER.claimed(_account);
    if (alreadyClaimed > 0) {
        revert TeamElectionWouldBlackoutClaims(_account, alreadyClaimed);
    }
    VESTER.wlfiBurnAllocation(_account, amountToBurn);
    REGISTRY.wlfiBurnAllocation(_account, amountToBurn);
}
```

Equivalently, document the blackout behavior and add an invariant that asserts no team user with `claimed > 0` is elected during deployment. The script's existing `invariants` block should add a case that pre-claims under a team category and then asserts the post-election claim trajectory.

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.2.6 Legacy retail template is copied into category 45 with no validation of percentage or end timestamp

Description: The deploy script reads the existing template from category 1 and copies it verbatim into category 45 at index 0, then appends the new `2 + 2` template at index 1. The only validation is that exactly one old template exists.

```
//deploy-wlfi-v3-updates.ts#L148-L181
const oldTemplates = await vesterProxy.getAllCategoryTemplates(1);
if (oldTemplates.length !== 1) {
  throw new Error('Expected old templates to have length 1');
}

transactions.push(
  // Bring over the old template
  await prettyPrintEncodedDataWithTypeSafety(
    core,
    'WorldLibertyFinancialVester',
    vesterProxy.ownerSetCategoryTemplate,
    [
      FinalizedVestingCategory.RETAIL, 0, {
      percentageOfAllocation: oldTemplates[0].percentageOfAllocation,
```

```
      startTimestamp: oldTemplates[0].startTimestamp,
      cliffTimestamp: oldTemplates[0].cliffTimestamp,
      endTimestamp: oldTemplates[0].endTimestamp,
    },
    ],
  ),
  // Put in 2 + 2
  await prettyPrintEncodedDataWithTypeSafety(
    core,
    'WorldLibertyFinancialVester',
    vesterProxy.ownerSetCategoryTemplate,
    [
      FinalizedVestingCategory.RETAIL, 1, {
      percentageOfAllocation: parseEther(`${0.8}`),
      startTimestamp: VESTING_START_TIMESTAMP,
      cliffTimestamp: VESTING_START_TIMESTAMP,
      endTimestamp: VESTING_START_TIMESTAMP + TWO_YEARS_IN_SECONDS,
    },
    ],
  ),
```

The vester walks templates in order, capping each segment by the remaining allocation:

```
//WorldLibertyFinancialVester.sol#L298-L336
function _unlockedTotal(
    VesterStorage storage $,
    uint8 _category,
    uint112 _allocation
) internal view returns (uint256) {
    ...
    for (uint8 i; i < count; ) {
        Template memory template = $.categoryTemplates[_category][i];

        uint256 segmentCap = _allocation * uint256(template.percentageOfAllocation) / MAX_PERCENTAGE;
        segmentCap = segmentCap < remainingCap ? segmentCap : remainingCap;
        ...
    }
    return totalUnlocked;
}
```

The behavior of category 45 after deployment depends on three properties of `oldTemplates[0]` that the script never asserts:

1. `percentageOfAllocation`. If `oldTemplates[0].percentageOfAllocation + 0.8 ether > 1 ether`, template 1 is silently capped by `remainingCap` and unlocks less than 80% of allocation. If `oldTemplates[0].percentageOfAllocation == 0`, only 80% of allocation ever vests through category 45, and 20% becomes permanently unreachable through this category.
2. `endTimestamp`. If `oldTemplates[0].endTimestamp > VESTING_START_TIMESTAMP`, the post-election retail schedule is a hybrid of the legacy curve and the new 2-year curve, not the proposal's clean "2-year cliff + 2-year linear" curve.
3. `cliffTimestamp` relative to `block.timestamp`. If the old template was configured with `start < cliff`, it interacts with the phantom-unlock issue described in M-02 of this audit.

The script does not print `oldTemplates[0]` and the multisig signers have no opportunity to eyeball it before approving the bundle.

Impact: The script's correctness for the retail cohort is fully delegated to whatever happens to be stored at `categoryTemplates[1][0]` on mainnet at deployment time. If that template diverges from the assumed shape, the post-election retail vesting curve quietly diverges from the proposal in a way that is not visible from the script source, the proposal text, or the `invariants` block. The most material divergence is silent capping: a template

0 with `percentageOfAllocation > 0.2 ether` causes template 1 to be capped at `1 ether - oldPct`, so retail receives less than the advertised 80% on the new schedule.

Proof of Concept:

```
Assume mainnet category 1 currently holds:
  oldTemplates[0] = {
    percentageOfAllocation: 0.5 ether,    // 50%
    startTimestamp:         someTime,
    cliffTimestamp:         someTime,
    endTimestamp:           someTimeInPast
  }

After the script runs, category 45 holds:
  index 0 = oldTemplates[0]                    // 50% allocation, fully unlocked
  index 1 = { 0.8 ether, VESTING_START, ... } // intends 80% over 2y

For a user with allocation = 1000 WLFI:
  remainingCap starts at 1000.
  index 0: segmentCap = 1000 * 0.5 = 500. Already past endTimestamp, fully unlocked.
           remainingCap -= 500 -> 500.
  index 1: segmentCap = 1000 * 0.8 = 800, capped to remainingCap = 500.
           Linear from VESTING_START over 2y, max 500 WLFI.

Effective curve: 500 WLFI immediately claimable post-election, 500 WLFI over 2y.
Proposal curve: 0 immediately, then linear to 800 WLFI over 2y, with the
remaining 200 (old template) already accounted for in claimed.

The old percentage is unknown to anyone reading this script in isolation,
so the deviation is invisible.
```

Recommended Mitigation: Add explicit assertions in the script before the template copy, with concrete expected values pulled from the on-chain state of `categoryTemplates[1][0]` at the time the proposal was drafted. For example:

```
const old = oldTemplates[0];
if (old.percentageOfAllocation !== EXPECTED_LEGACY_RETAIL_PCT) {
  throw new Error(`Unexpected legacy retail percentage: ${old.percentageOfAllocation}`);
}
if (old.endTimestamp > VESTING_START_TIMESTAMP) {
  throw new Error(`Legacy retail template still active past VESTING_START`);
}
if (old.percentageOfAllocation + parseEther('0.8') > parseEther('1')) {
  throw new Error('Legacy retail percentage + new 80% exceeds 100%');
}
```

Also extend the `invariants` block to print and assert the resulting `categoryTemplates[45]` shape so the multisig has a final on-chain check before the bundle executes.

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.2.7 `WorldLibertyFinancialVester::ownerSetCategoryTemplate` does not enforce sum of `percentageOfAllocation` across templates, allowing misconfiguration to permanently strand user allocation

Description: `WorldLibertyFinancialVester::ownerSetCategoryTemplate` performs index-bound and timestamp-ordering checks but does not constrain `percentageOfAllocation` bounds, nor does it validate that the sum of percentages across all templates in a category equals (or is) `MAX_PERCENTAGE (= 1 ether)`:

```
// WorldLibertyFinancialVester.sol
function ownerSetCategoryTemplate(
    uint8 _category,
```

```
    uint8 _index,
    Template calldata _template
) external onlyWorldLibertyOwner(msg.sender) {
    if (_index >= MAX_TEMPLATE_COUNT) { revert InvalidTemplateCount(); }
    if (_template.endTimestamp != 0) {
        if (_template.startTimestamp > _template.cliffTimestamp
            || _template.cliffTimestamp > _template.endTimestamp) {
            revert InvalidTemplateTimestamp();
        }
    }
    VesterStorage storage $ = _getStorage();
    $.categoryTemplates[_category][_index] = _template; // @audit accepts any percentage; no sum check
    uint8 count = $.categoryInfo[_category].templateCount;
    if (_index >= count) {
        count = _index + 1;
        $.categoryInfo[_category].templateCount = count;
        ...
    }
    ...
}
```

The `_unlockedTotal` math in the same contract uses a `remainingCap` mechanic that clamps the total unlocked amount at `allocation` when `sum(percentages) > 1 ether`. Concretely, if the sum is `< 1 ether`, the missing fraction `(1 ether - sum)` of every user's allocation becomes **permanently unclaimable** — `_unlockedTotal` never produces an `unlocked` value exceeding `sum * allocation / 1 ether`, so `claimable = unlocked - claimed` caps out strictly below `allocation`.

```
// WorldLibertyFinancialVester.sol
function _unlockedTotal(
    VesterStorage storage $,
    uint8 _category,
    uint112 _allocation
) internal view returns (uint256) {
    ...
    uint256 remainingCap = _allocation;
    for (uint8 i; i < count; ) {
        Template memory template = $.categoryTemplates[_category][i];
        uint256 segmentCap = _allocation * uint256(template.percentageOfAllocation) / MAX_PERCENTAGE;
        segmentCap = segmentCap < remainingCap ? segmentCap : remainingCap;
        if (segmentCap != 0) {
            uint256 unlocked = _segmentUnlocked(template, segmentCap);
            totalUnlocked += unlocked;
            remainingCap -= segmentCap; // @audit if sum<1e18, loop ends with remainingCap > 0
            ↪  permanently
            if (remainingCap == 0) { break; }
        }
        unchecked { ++i; }
    }
    return totalUnlocked;
}
```

Impact: Any owner action that writes a template resulting in a category's total coverage `< 1 ether` permanently strands `(1 ether - sum) * allocation / 1 ether` of every user's allocation activated under that category. `ownerSetCategoryTemplate` is the ongoing admin surface for category configuration — every single-slot write, every future update, and every fix after V3 ships goes through this function without any safety net.

Proof of Concept: Run the following test:

```
import { loadFixture, time } from '@nomicfoundation/hardhat-network-helpers';
import { expect } from 'chai';
import { parseEther } from 'ethers';
import { impersonateReq } from '../../script/execution-utils';
```

```
import { ONE_ETH_BI, ZERO_BI } from '../../src/no-dependencies-constants';
import { IWorldLibertyFinancialVester } from '../../src/types';
import { FinalizedVestingCategory, WLFI_START_TIMESTAMP } from '../../src/wlfi-constants';
import { deployWlfiV2Fixture } from '../fixtures';
import { expectEvent, expectThrowWithCustomError } from '../utils';
import { advanceTimeToAfterStartTimestamp } from '../wlfi/wlfi-utils';

type Ctx = Awaited<ReturnType<typeof deployWlfiV2Fixture>>;
let ctx: Ctx;

const DEFAULT_AMOUNT = parseEther('100');
const DEFAULT_CATEGORY = 2n;

// An immediate-unlock template: endTimestamp = 0 means "fully unlocked regardless of time"
const IMMEDIATE_TEMPLATE: IWorldLibertyFinancialVester.TemplateStruct = {
  percentageOfAllocation: parseEther('1'),
  startTimestamp: ZERO_BI,
  cliffTimestamp: ZERO_BI,
  endTimestamp: ZERO_BI,
};

describe('WorldLibertyFinancialVester - Edge Cases (audit)', () => {
  beforeEach(async () => {
    ctx = await loadFixture(deployWlfiV2Fixture);
  });

  describe('ownerSetCategoryTemplate sum-of-percentages gap', () => {
    it('permanently strands allocation when sum(percentageOfAllocation) < 1 ether', async function () {
      const wlfi = await impersonateReq(ctx.wlfi, true);
      const user = ctx.core.hhUser1;

      // Configure cat 2 with ONLY 50% coverage (NO second template to fill the rest)
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(Number(DEFAULT_CATEGORY), true);
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryTemplate(Number(DEFAULT_CATEGORY), 0, {
        percentageOfAllocation: parseEther('0.5'),   // 50%
        startTimestamp: ZERO_BI,
        cliffTimestamp: ZERO_BI,
        endTimestamp: ZERO_BI,   // immediate unlock
      });

      // Activate user with 100 WLFI
      await ctx.wlfi.connect(ctx.wlfiOwner).transfer(user.address, DEFAULT_AMOUNT);
      await ctx.wlfi.connect(user).approve(ctx.vester, DEFAULT_AMOUNT);
      await ctx.vester.connect(wlfi).wlfiActivateVest(user.address, Number(DEFAULT_CATEGORY),
      ↪  DEFAULT_AMOUNT);

      await advanceTimeToAfterStartTimestamp(ctx);

      // Claimable should be capped at 50% of allocation
      expect(await ctx.vester.claimable(user.address)).to.eq(DEFAULT_AMOUNT / 2n);

      // User claims max
      await ctx.vester.connect(wlfi).wlfiClaimFor(user.address);

      // 50% has been claimed; the other 50% is PERMANENTLY STRANDED in Vester
      expect(await ctx.vester.claimed(user.address)).to.eq(DEFAULT_AMOUNT / 2n);
      expect(await ctx.vester.unclaimed(user.address)).to.eq(DEFAULT_AMOUNT / 2n);

      // Subsequent claim reverts with NothingToClaim - proof the remaining 50% is unreachable
      await expectThrowWithCustomError(
        ctx.vester.connect(wlfi).wlfiClaimFor(user.address),
        ctx.vester,
        'NothingToClaim',
```

```
      );

      // Even after a long time, no new unlock - template math caps at 50%
      await time.increase(86_400 * 365 * 10); // 10 years
      await expectThrowWithCustomError(
        ctx.vester.connect(wlfi).wlfiClaimFor(user.address),
        ctx.vester,
        'NothingToClaim',
      );
      expect(await ctx.vester.unclaimed(user.address)).to.eq(DEFAULT_AMOUNT / 2n);
    });
  });

});
```

Recommended Mitigation: Consider enforcing the sum invariant inside `ownerSetCategoryTemplate`.

```
function ownerSetCategoryTemplate(
    uint8 _category,
    uint8 _index,
    Template calldata _template
) external onlyWorldLibertyOwner(msg.sender) {

 ...
 VesterStorage storage $ = _getStorage();
    $.categoryTemplates[_category][_index] = _template;

    uint8 count = $.categoryInfo[_category].templateCount;
    if (_index >= count) {
        count = _index + 1;
        $.categoryInfo[_category].templateCount = count;
        emit SetCategoryTemplateCount(_category, count);

++    uint256 total;
++    for (uint8 i; i < count; i++) {
++        total += $.categoryTemplates[_category][i].percentageOfAllocation;
++   }
++    if (total > MAX_PERCENTAGE) {
++       revert InvalidTemplateSum();
++    }
        emit SetCategoryTemplate(_category, _index, _template);
    }
}
```

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.2.8 `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` overwrite resets `isActivated=false`, freezing the user's WLFI balance via V2's `_update` gate

Description: `WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` unconditionally writes the full `LegacyUser` struct with `isActivated: false` hardcoded, regardless of whether an entry for that user already exists:

```
//WorldLibertyFinancialRegistry.sol
$.legacyUserMap[_users[i]] = LegacyUser({
    amount: uint112(_amounts[i]),
    category: _categories[i],
    isActivated: false  // @audit hardcoded - overwrites any prior activation
});
```

If a user has activated, claimed some amount (so their WLFI balance is non-zero again), and the agent subsequently re-inserts them — the balance-match check `WLFI.balanceOf(user) == _amounts[i]` passes (user's post-claim balance is non-zero), and the overwrite resets `isActivated` to `false`.

This state is toxic because `WorldLibertyFinancialV2::_update` refuses to transfer to/from a legacy-user-not-activated account after trading-start:

```
// File: contracts/wlfi/WorldLibertyFinancialV2.sol
function _update(address _from, address _to, uint256 _value)
    notBlacklisted(_msgSender()) notBlacklisted(_from) notBlacklisted(_to)
    internal override(...) {
    if (_to == address(this)) { revert InvalidAccount(); }
    if (!isAfterTradingStartTimestamp()) { ... }

    if (REGISTRY.isLegacyUserAndIsNotActivated(_from) && _msgSender() != owner()) {
        revert AccountNotActivated(_from); // @audit fires for the re-inserted user
    }
    if (REGISTRY.isLegacyUserAndIsNotActivated(_to)) {
        revert AccountNotActivated(_to);
    }
    return super._update(_from, _to, _value);
}
```

Once re-inserted, the user:

1. Cannot transfer their own WLFI (reverts `AccountNotActivated(user)` at line 381).
2. Cannot claim further from the Vester (`VESTER.wlfiClaimFor` internally calls `IERC20(WLFI).safeTransfer(user, claimable)`, which trips the same `_update` check with `_to = user`).
3. Cannot be elected via `WorldLibertyFinancialV3::_electVestingUpdate` (V3 gates on `!REGISTRY.isLegacyUserAndIsNotActivated(_account)`).

The only escape is `WorldLibertyFinancialV2::ownerActivateAccount(user, _bypassVester=true)` — owner must intervene manually to flip `isActivated` back to true without pulling tokens into the Vester (the user cannot be re-pulled because `VESTER.users[user].initialized` is still true from their original activation). Post-recovery the Registry's `amount` no longer matches the Vester's `allocation`, creating a permanent state drift.

Impact: The whitelist agent can unilaterally DoS the claims and liquidity of any activated user who has claimed at least 1 wei. Victims lose access to their own WLFI tokens and cannot participate in the V3 election. Scale scales linearly with agent batch size — a single `agentBulkInsertLegacyUsers` call with 500 victim addresses freezes 500 users in one transaction.

An honest agent re-running a batch they previously submitted would unintentionally de-activate any user whose balance has changed since the first run.

Proof of Concept: Run the following tests:

```
import { loadFixture } from '@nomicfoundation/hardhat-network-helpers';
import { expect } from 'chai';
import { parseEther } from 'ethers';
import { impersonateReq } from '../../script/execution-utils';
import { ONE_ETH_BI, ZERO_BI } from '../../src/no-dependencies-constants';
import { IWorldLibertyFinancialVester } from '../../src/types';
import { FinalizedVestingCategory, WLFI_START_TIMESTAMP } from '../../src/wlfi-constants';
import { deployWlfiV2Fixture } from '../fixtures';
import { expectEvent, expectThrowWithCustomError } from '../utils';
import { advanceTimeToAfterStartTimestamp } from '../wlfi/wlfi-utils';

type Ctx = Awaited<ReturnType<typeof deployWlfiV2Fixture>>;
let ctx: Ctx;

const DEFAULT_AMOUNT = parseEther('100');
```

```
const IMMEDIATE_TEMPLATE: IWorldLibertyFinancialVester.TemplateStruct = {
  percentageOfAllocation: parseEther('1'),
  startTimestamp: ZERO_BI,
  cliffTimestamp: ZERO_BI,
  endTimestamp: ZERO_BI,
};

describe('WorldLibertyFinancialRegistry - Edge Cases (audit)', () => {
  beforeEach(async () => {
    ctx = await loadFixture(deployWlfiV2Fixture);
  });


  describe('agentBulkInsertLegacyUsers  overwrite activated user (Finding
  ↪  [*`WorldLibertyFinancialRegistry::agentBulkInsertLegacyUsers` can be griefed by dust transfers
  ↪  when WLFI is transferable*](#worldlibertyfinancialregistryagentbulkinsertlegacyusers-can-be-grief
  ↪  ed-by-dust-transfers-when-wlfi-is-transferable))', () => {
    it('resets isActivated=false when re-inserting a user who has claimed partial allocation', async
    ↪  function () {
      // activate user normally with cat=1 (retail-like category)
      // Use hhUser2 so initial balance is zero and balance-match check aligns.
      const user = ctx.core.hhUser2;
      const category = 1;

      // Enable cat 1 with immediate-unlock template so user can claim something
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(category, true);
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryTemplate(category, 0, IMMEDIATE_TEMPLATE);

      await ctx.wlfi.connect(ctx.wlfiOwner).transfer(user.address, DEFAULT_AMOUNT);
      const nonce0 = await ctx.registry.nonce();
      await ctx.registry.connect(ctx.wlfiOwner).agentBulkInsertLegacyUsers(
        nonce0,
        [user.address],
        [DEFAULT_AMOUNT],
        [category],
      );
      await ctx.wlfi.connect(ctx.wlfiOwner).ownerActivateAccount(user.address, false);
      expect(await ctx.registry.isLegacyUserAndIsActivated(user.address)).to.be.true;

      // User claims all their allocation (user now holds 100 WLFI again)
      await advanceTimeToAfterStartTimestamp(ctx);
      await ctx.wlfi.connect(user).claimVest();
      expect(await ctx.wlfi.balanceOf(user.address)).to.eq(DEFAULT_AMOUNT);

      // agent re-inserts the SAME user with their current balance
      const nonce1 = await ctx.registry.nonce();
      await ctx.registry.connect(ctx.wlfiOwner).agentBulkInsertLegacyUsers(
        nonce1,
        [user.address],
        [DEFAULT_AMOUNT],
        [category],
      );

      // isActivated is forcibly reset to false
      expect(await ctx.registry.isLegacyUserAndIsActivated(user.address)).to.be.false;
    });

    it('subsequent transfers from the re-inserted user revert with AccountNotActivated', async function
    ↪  () {
      // Follow-on effect: the V2._update check blocks transfers to/from
      // a legacy-user-not-activated account → user's balance is effectively frozen
      const user = ctx.core.hhUser2;
      const category = 1;
```

```
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryEnabled(category, true);
      await ctx.vester.connect(ctx.wlfiOwner).ownerSetCategoryTemplate(category, 0, IMMEDIATE_TEMPLATE);

      await ctx.wlfi.connect(ctx.wlfiOwner).transfer(user.address, DEFAULT_AMOUNT);
      const nonce0 = await ctx.registry.nonce();
      await ctx.registry.connect(ctx.wlfiOwner).agentBulkInsertLegacyUsers(
        nonce0,
        [user.address],
        [DEFAULT_AMOUNT],
        [category],
      );
      await ctx.wlfi.connect(ctx.wlfiOwner).ownerActivateAccount(user.address, false);
      await advanceTimeToAfterStartTimestamp(ctx);
      await ctx.wlfi.connect(user).claimVest();

      // Re-insert → user is now "legacy-not-activated" again
      const nonce1 = await ctx.registry.nonce();
      await ctx.registry.connect(ctx.wlfiOwner).agentBulkInsertLegacyUsers(
        nonce1,
        [user.address],
        [DEFAULT_AMOUNT],
        [category],
      );

      // User's attempt to transfer their own tokens reverts
      const recipient = ctx.core.hhUser3;
      await expectThrowWithCustomError(
        ctx.wlfi.connect(user).transfer(recipient.address, 1n),
        ctx.wlfi,
        'AccountNotActivated',
        user.address,
      );
    });
  });
});
```

Recommended Mitigation: Consider rejecting overwrites of existing entries in `agentBulkInsertLegacyUsers`. This matches the mental model that Registry entries are set once per user at seeding time.

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.3 Informational

7.3.1 Duplicated `WLFI` access control check can be extracted into a modifier

Description: Both `WorldLibertyFinancialRegistry` and `WorldLibertyFinancialVester` gate their `wlfi`-prefixed external functions with an identical inline access control check that reverts when the caller is not the `WLFI` contract. The same three-line block is duplicated across four functions in the registry and five functions in the vester.

`WorldLibertyFinancialRegistry`: `wlfiActivateAccount`, `wlfiBurnAllocation`, `wlfiReallocateFrom`, `wlfiSetCategory`.

```
// WorldLibertyFinancialRegistry.sol#L34-L90
function wlfiActivateAccount(address _user) external {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}

function wlfiBurnAllocation(address _user, uint256 _amount) external {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}

function wlfiReallocateFrom(address _from, address _to) external {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}

function wlfiSetCategory(address _user, uint8 _category) external {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}
```

`WorldLibertyFinancialVester`: `wlfiActivateVest`, `wlfiBurnAllocation`, `wlfiClaimFor`, `wlfiReallocateFrom`, `wlfiSetCategory`.

```
//WorldLibertyFinancialVester.sol#L95-L173
function wlfiActivateVest(address _user, uint8 _category, uint112 _amount) external whenNotPaused {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}

function wlfiBurnAllocation(address _user, uint256 _amount) external whenNotPaused {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}

function wlfiClaimFor(address _user) external whenNotPaused returns (uint256) {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
```

```
    ...
}

function wlfiReallocateFrom(address _from, address _to) external whenNotPaused {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}

function wlfiSetCategory(address _user, uint8 _category) external whenNotPaused {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    ...
}
```

Impact: Code duplication. A future change to the authorization rule requires editing nine call sites across two contracts, which increases the risk of inconsistent updates.

Recommended Mitigation: Extract the check into a modifier and apply it to each `wlfi`-prefixed function in both contracts.

```
modifier onlyWLFI() {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }
    _;
}

// Registry
function wlfiActivateAccount(address _user) external onlyWLFI {
    _activateAccount(_user);
}

function wlfiBurnAllocation(address _user, uint256 _amount) external onlyWLFI {
    ...
}

function wlfiReallocateFrom(address _from, address _to) external onlyWLFI {
    ...
}

function wlfiSetCategory(address _user, uint8 _category) external onlyWLFI {
    ...
}

// Vester
function wlfiActivateVest(address _user, uint8 _category, uint112 _amount) external onlyWLFI
↪   whenNotPaused {
    _activateVest(_user, _category, _amount);
}

function wlfiBurnAllocation(address _user, uint256 _amount) external onlyWLFI whenNotPaused {
    ...
}

function wlfiClaimFor(address _user) external onlyWLFI whenNotPaused returns (uint256) {
    return _claim(_user);
}

function wlfiReallocateFrom(address _from, address _to) external onlyWLFI whenNotPaused {
    ...
```

```
}

function wlfiSetCategory(address _user, uint8 _category) external onlyWLFI whenNotPaused {
    ...
}
```

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.3.2 `WorldLibertyFinancialRegistry::wlfiBurnAllocation` leaves stale legacy-user storage and emits the wrong event when the full allocation is burned

Description: `WorldLibertyFinancialRegistry::wlfiBurnAllocation` decrements `userInfo.amount` by `_amount` and emits `LegacyUserUpdated`. When `_amount` equals the user's full remaining allocation, the entry is left partially populated in storage (`amount = 0`, but `category` and `isActivated` retain their previous values), and no `LegacyUserRemoved` event is emitted.

```
function wlfiBurnAllocation(address _user, uint256 _amount) external {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }

    RegistryStorage storage $ = _getStorage();
    LegacyUser storage userInfo = $.legacyUserMap[_user];
    userInfo.amount -= uint112(_amount); // safe cast

    emit LegacyUserUpdated(_user, userInfo.amount, userInfo.category, userInfo.isActivated);
}
```

The on-chain view functions are unaffected because every reader gates on `amount != 0`:

- `isLegacyUser` (L157-L162)
- `isLegacyUserAndIsActivated` / `isLegacyUserAndIsNotActivated` (L164-L178)
- `_validateUserAndReturn` (L209-L220)

So from the perspective of on-chain consumers, the user ceases to be a legacy user the moment `amount` reaches zero, and a subsequent `agentBulkInsertLegacyUsers` cleanly overwrites the slot with `isActivated: false` (L115-L119). The concern is off-chain: the contract defines a dedicated `LegacyUserRemoved` event and emits it from `agentBulkRemoveLegacyUsers` (L145) but not from the burn path, even though burning to zero is semantically the same state transition.

Impact: Indexers or monitoring systems that track the legacy-user set by listening to `LegacyUserRemoved` will miss users whose allocations are fully burned, producing a divergent off-chain view of legacy-user membership. The caller also misses the storage-clear gas refund that `agentBulkRemoveLegacyUsers` and `wlfiReallocateFrom` obtain via `delete`.

Recommended Mitigation: When the post-decrement amount is zero, delete the entry and emit `LegacyUserRemoved` instead of `LegacyUserUpdated`.

```
function wlfiBurnAllocation(address _user, uint256 _amount) external {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }

    RegistryStorage storage $ = _getStorage();
    LegacyUser storage userInfo = $.legacyUserMap[_user];
    uint112 newAmount = userInfo.amount - uint112(_amount); // safe cast

    if (newAmount == 0) {
        delete $.legacyUserMap[_user];
```

```
        emit LegacyUserRemoved(_user);
    } else {
        userInfo.amount = newAmount;
        emit LegacyUserUpdated(_user, newAmount, userInfo.category, userInfo.isActivated);
    }
}
```

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.3.3 `WorldLibertyFinancialV3::_electVestingUpdate` hard-codes "not category 1 → team" and hard-codes magic category numbers

Description: The election branch in `WorldLibertyFinancialV3._electVestingUpdate` partitions the entire legacy-user population on the literal `oldCategory == 1`:

```
// WorldLibertyFinancialV3.sol#L85-L96
uint8 oldCategory = REGISTRY.getLegacyUserCategory(_account);
if (oldCategory == 45 || oldCategory == 47) {
    revert ElectionAlreadyPerformed();
}

uint8 newCategory;
if (oldCategory == 1) {
    newCategory = 45;
} else {
    // assert(oldCategory != 1); // To make clear this else branch should cover non category 1 vesting
    newCategory = 47;
}
```

There are two related concerns here:

1. **Magic numbers.** `1`, `45`, and `47` appear as raw literals inside the core election logic. The TypeScript side of the codebase defines `FinalizedVestingCategory.RETAIL = 45` and `FinalizedVestingCategory.TEAM = 47` (src/wlfi-constants.ts:34), but the Solidity side uses bare integers with no in-contract constants, interfaces-level documentation, or even a `uint8 private constant RETAIL_CATEGORY = 45` declaration. Reviewers and future upgraders must cross-reference external files to understand what each literal means. The same literals appear in `IWorldLibertyFinancialV3` and `WorldLibertyFinancialV3` without a shared source of truth.
2. **Two-way partition of every conceivable category.** The `else` branch catches *every* category that is not exactly `1`, `45`, or `47`, and sends those users to team (47). This includes:
   - Legitimate team/advisor/partner buckets (categories 2..20 as enabled in the deploy script).
   - Categories the project has not yet minted (21..44, 46, 48..255).
   - `category = 0` if the registry ever has an entry with `amount != 0` but `category == 0`. `agentBulkInsertLegacyUsers` does not reject `category == 0`:

```
    for (uint256 i; i < _users.length; ++i) {
        if (_users[i] == address(0) || _amounts[i] == 0) {
            revert InvalidBulkInsertLegacyUserAtIndex(i);
        }
        ...
        $.legacyUserMap[_users[i]] = LegacyUser({
            amount: uint112(_amounts[i]),
            category: _categories[i],
            isActivated: false
        });
    }
```

and `wlfiSetCategory` does not reject `_category == 0` either. A user ever inserted with `_category == 0`, and subsequently bypass-activated by the owner, would be elected straight to team.

Per the proposal, early supporters are a distinct cohort from founders/team/advisors/partners, so it is likely that the intent is specifically "category 1 = early supporter → retail, everything else in the current retail/team scope → team". But that intent is not expressed defensively: if the registry acquires any category that should not be eligible for team (e.g. a future non-locked cohort, an accidentally-zero category), it is silently mapped to team.

Impact: The code works correctly on the expected inputs (categories 1 and 2..20 with team/advisor/partner semantics). The concern is maintainability and defensive behavior for future expansions of the registry and for operator error in bulk-insert.

Recommended Mitigation:

- Introduce named constants in the contract (or in `IWorldLibertyFinancialV3`):

```
uint8 private constant EARLY_SUPPORTER_CATEGORY = 1;
uint8 public constant RETAIL_CATEGORY  = 45;
uint8 public constant TEAM_CATEGORY    = 47;
```

  and replace every literal with the constant. Doing so matches the TypeScript-side `FinalizedVestingCategory` enum and documents the semantics at the call site.

- Make the partition explicit. For example, require that `oldCategory` is a member of a known "team-like" set before mapping to 47:

```
uint8 oldCategory = REGISTRY.getLegacyUserCategory(_account);
if (oldCategory == RETAIL_CATEGORY || oldCategory == TEAM_CATEGORY) {
    revert ElectionAlreadyPerformed();
}

uint8 newCategory;
if (oldCategory == EARLY_SUPPORTER_CATEGORY) {
    newCategory = RETAIL_CATEGORY;
} else if (isKnownTeamCategory(oldCategory)) {
    newCategory = TEAM_CATEGORY;
} else {
    revert UnknownCategory(oldCategory);
}
```

  Where `isKnownTeamCategory` enumerates the exact categories the proposal intends to migrate to team.

- Reject `_category == 0` in `agentBulkInsertLegacyUsers`, `wlfiSetCategory`, `ownerSetCategoryEnabled`, and `ownerSetCategoryTemplate` (see also the related concerns captured in the separate template-percentage note below).

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.3.4 Redundant `assert` in `WorldLibertyFinancialV3::_electVestingUpdate` after self-write

Description: At the end of `WorldLibertyFinancialV3::_electVestingUpdate`, the function asserts that the registry now has the new `category` that the same transaction just wrote two lines above:

```
VESTER.wlfiSetCategory(_account, newCategory);
REGISTRY.wlfiSetCategory(_account, newCategory);

assert(REGISTRY.getLegacyUserCategory(_account) == newCategory);

emit VestingUpdated(_account, oldCategory, newCategory);
```

`REGISTRY.wlfiSetCategory` unconditionally executes `userInfo.category = _category`, so the postcondition cannot fail under normal control flow:

```
function wlfiSetCategory(address _user, uint8 _category) external {
    if (msg.sender != address(WLFI)) {
        revert Unauthorized();
    }

    RegistryStorage storage $ = _getStorage();
    LegacyUser storage userInfo = $.legacyUserMap[_user];
    userInfo.category = _category;

    emit LegacyUserUpdated(_user, userInfo.amount, _category, userInfo.isActivated);
}
```

The only realistic way the assert can trigger is a malicious or upgraded registry implementation, in which case the assert cannot protect the user anyway. The check pays an external call and a storage read every election to verify a value the same transaction just wrote.

Recommended Mitigation: Remove the assert.

```
     VESTER.wlfiSetCategory(_account, newCategory);
     REGISTRY.wlfiSetCategory(_account, newCategory);

-    assert(REGISTRY.getLegacyUserCategory(_account) == newCategory);
     emit VestingUpdated(_account, oldCategory, newCategory);
```

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.3.5 `assert` is used to enforce user-reachable invariants in `WorldLibertyFinancialVester`

Description: Solidity's `assert` is documented as a check for conditions that should never be false and triggers a `Panic(0x01)` when it fails. The vester uses `assert` in several places where the condition is actually reachable through normal user or admin actions, not only through impossible internal states:

```
// WorldLibertyFinancialVester.sol#L108-L114
userInfo.allocation -= uint112(_amount); // safe cast
assert(userInfo.claimed <= userInfo.allocation);

$.totalAllocated -= uint112(_amount);
assert($.totalClaimed <= $.totalAllocated);
```

```
// WorldLibertyFinancialVester.sol#L303
assert(_allocation != 0); // This should never be 0
```

```
// WorldLibertyFinancialVester.sol#L341
assert(_segmentCap != 0);
```

```
// WorldLibertyFinancialVester.sol#L362
assert(span != 0);
```

The `assert(userInfo.claimed <= userInfo.allocation)` assertion at L111 is the most important: it fails whenever the caller asks to burn an amount that would push the user's new allocation below what they have already claimed (see L-03). That is a user-reachable condition — the user simply pre-claims under their old category — yet the failure mode is an unnamed panic rather than a descriptive revert.

`_unlockedTotal`'s `assert(_allocation != 0)` is also user-reachable through `claimable(user)` if a fully-burned user ever exists: `_claimable` gates on `!_userInfo.initialized` but not on `_userInfo.allocation == 0`. Under the current V3 logic the allocation cannot be zero while initialized is true (the only burn path burns at most 10%), but this is an implicit invariant held up by every caller rather than an invariant enforced at the function boundary.

Impact:

- Worse error surface: panics consume calldata-sized gas and leak no structured information, so operators and off-chain dashboards see a bare `Panic(0x01)` instead of a named error like `BurnExceedsUnclaimed(address,uint112,uint112)` or `AllocationIsZero(address)`.
- Harder to reason about: `assert` conventionally signals "can never happen". Using it for user-reachable conditions masks the fact that these are real edge cases that need explicit handling.

Recommended Mitigation: Replace the user-reachable asserts with named `reverts`. Example for `wlfiBurnAllocation` . For `_unlockedTotal`, prefer returning `0` over asserting when `_allocation == 0`, so that `claimable(user)` and `_claim(user)` cannot panic if the allocation is ever zeroed out:

```
function _unlockedTotal(
    VesterStorage storage $,
    uint8 _category,
    uint112 _allocation
) internal view returns (uint256) {
    if (_allocation == 0) {
        return 0;
    }
    uint8 count = $.categoryInfo[_category].templateCount;
    ...
}
```

Keep `assert` only for truly impossible invariants (e.g. proofs that a downstream library has already validated), and document why each remaining `assert` cannot be triggered by any caller-reachable state.

WLFI: Acknowledged.

7.3.6 WLFI pause does not block retail elections, only team elections and Vester-gated flows

Description: The V3 election flow interacts with two independent pause states — WorldLibertyFinancialV3 (pausable via ownerPause/guardianPause, inherited from V2) and WorldLibertyFinancialVester (pausable via ownerPause/guardianPause). Neither `electVestingUpdate` nor `ownerElectVestingUpdatesFor` has whenNotPaused on V3 itself; pause semantics are entirely downstream.

Downstream reach:

- `Vester.wlfiSetCategory` — whenNotPaused (line 158). Invoked for BOTH retail and team elections.
- `Vester.wlfiBurnAllocation` — whenNotPaused (line 103). Invoked only for team (cat 47) elections.
- `WLFI.burn` (via Vester's wlfiBurnAllocation) — routed through V2's _update which is ERC20PausableUpgradeable.whenNotPaused. Invoked only for team elections.

An operator who pauses WLFI during an incident expecting to halt all new elections will find retail elections still land, mutating Registry + Vester category state. Only pausing the Vester achieves a uniform halt.

Impact: A retail user with a valid signature can still flip their category to 45 while WLFI is paused.

Recommended Mitigation: Consider adding `whenNotPaused` check to the `WorldLibertyFinancialV3::electVestingUpdate` and `WorldLibertyFinancialV3::ownerElectVestingUpdatesFor` entry points.

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.3.7 `WorldLibertyFinancialV3::electVestingUpdate` signature has no nonce and no deadline, making replay protection purely state-based

Description: `WorldLibertyFinancialV3::electVestingUpdate` verifies an EIP-712 signature over `Election(address account)`. The struct contains only the `account` field — there is no `nonce`, no `deadline`, no

version marker:

```
// WorldLibertyFinancialV3.sol
bytes32 private constant ELECTION_TYPEHASH = keccak256("Election(address account)");

function electVestingUpdate(bytes calldata _signature) external {
    address account = _msgSender();
    bytes32 hash = _hashTypedDataV4(keccak256(abi.encode(ELECTION_TYPEHASH, account)));
    if (authorizedSigner() != ECDSA.recover(hash, _signature)) {
        revert InvalidSignature();
    }
    _electVestingUpdate(account);
}
```

Once a signature is issued by `authorizedSigner`, it remains valid indefinitely until either:

- The user's Registry category transitions to 45 or 47 — at which point `_electVestingUpdate` reverts with `ElectionAlreadyPerformed`, providing state-based single-use semantics.
- The owner rotates `authorizedSigner` via `ownerSetAuthorizedSigner` — at which point the old sig no longer recovers to the new authorized address and `ECDSA.recover` produces a mismatch.

Recommended Mitigation: Consider adding nonce and deadline to the `Election` typehash.

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.

7.4 Gas Optimization

7.4.1 Public view functions that are never called internally can be marked `external`

Description: `WorldLibertyFinancialRegistry` declares three view functions as `public` even though none of them are invoked from within the contract. Marking them `external` is cheaper because `external` reads calldata directly, whereas `public` must copy arguments to memory to satisfy the internal-call ABI.

```
// WorldLibertyFinancialRegistry.sol#L164-L182
function isLegacyUserAndIsActivated(address _user) public view returns (bool) {
    ...
}

function isLegacyUserAndIsNotActivated(address _user) public view returns (bool) {
    ...
}

function getLegacyUserInfo(address _user) public view returns (LegacyUser memory) {
    return _validateUserAndReturn(_user);
}
```

None of these three functions are called anywhere else in the contract. `isLegacyUser` (L157) is the only sibling view that must remain `public`, because it is invoked internally from `wlfiReallocateFrom` (L64, L67) and `agentBulkRemoveLegacyUsers` (L140).

Impact: Minor gas overhead on every external call to these getters, and a small consistency issue — the other getters in the same section (`nonce`, `getLegacyUserCategory`, `getLegacyUserAllocation`) are already declared `external`.

Recommended Mitigation: Change the visibility of the three functions listed above from `public` to `external`.

```
function isLegacyUserAndIsActivated(address _user) external view returns (bool) { ... }
function isLegacyUserAndIsNotActivated(address _user) external view returns (bool) { ... }
function getLegacyUserInfo(address _user) external view returns (LegacyUser memory) { ... }
```

Leave `isLegacyUser` as `public` since it is consumed internally.

WLFI: Fixed in commit 1430e24.

Cyfrin: Verified.